

First 50 Zoho DSA Q?A

1. Replace every element with the greatest element on right side

Last Updated: 06-05-2019

Given an array of integers, replace every element with the next greatest element (greatest element on the right side) in the array. Since there is no element next to the last element, replace it with -1. For example, if the array is {16, 17, 4, 3, 5, 2}, then it should be modified to {17, 5, 5, 5, 2, -1}.

```
public class ReplaceWithNextGreatest {
    public static void main(String[] args) {
        int[] arr = {16, 17, 4, 3, 5, 2};
        replaceWithNextGreatest(arr);
        // Printing the modified array
        for (int num : arr) {
            System.out.print(num + " ");
        }
    }

    public static void replaceWithNextGreatest(int[] arr) {
        int n = arr.length;
        // Initialize the rightmost element as -1
        int maxRight = -1;

        // Traverse the array from right to left
        for (int i = n - 1; i >= 0; i--) {
            // Store the current element
            int temp = arr[i];
            // Replace current element with the maximum element on its right
            arr[i] = maxRight;
            // Update the maximum element on its right if necessary
            maxRight = Math.max(maxRight, temp);
        }
    }
}
```

2. Given a Boolean matrix mat[M][N] of size M X N, modify it such that if a matrix cell mat[i][j] is 1 then make its adjacent cells as 0.

```
import java.util.Arrays;

public class zoho_2_djacent {
    public static void main(String[] args) {
        int arr[][] = {
            {1, 0, 1, 1},
            {0, 0, 1, 0},
            {0, 0, 0, 0}
        };
    }
}
```

```

    };
    print(arr);
    System.out.println();
    modified(arr);

} static void modified(int arr[][]){
    for(int i=0;i< arr.length;i++){
        for(int j=0;j<arr[0].length;j++){
            if(arr[i][j]==1){
                if(i>0) arr[i-1][j]=0;
                if(i<arr.length-1) arr[i+1][j]=0;
                if(j>0) arr[i][j-1]=0;
                if(j< arr[0].length-1) arr[i][j+1]=0;
            }
        }
    }
    print(arr);
}
static void print(int arr[][]){
    for (int[] a : arr) {
        System.out.println(Arrays.toString(a));
    }
}
}

```

3. Equilibrium index of an array is an index such that the sum of elements at lower indexes is equal to the sum of elements at higher indexes. For example, in an array A:

Example :

Input: A[] = {-7, 1, 5, 2, -4, 3, 0}

Output: 3

```

public class Zoho3Equalibrium {
    public static void main(String[] args) {
        int ans[]={1, 2, 5, 2, 1};

        naive(ans);
//        System.out.println(Better(ans));
    }static void naive(int ans[]){
        int anx=0;
        for(int i=0;i<=ans.length-2;i++){
            int x=0;
            for(int j=0;j<i;j++){
                x+=ans[j];
            }
            int y=0;
            for(int j=i+1;j<ans.length;j++){
                y+=ans[j];
            }
            if(x==y) anx=i;
        }
    }
}

```

```

        System.out.println(ans);
    }
    static int Better(int ans[]) {
        int totalsum=0;
        int anss=0;
        int leftsum=0;
        for(int a:ans){
            totalsum+=a;
        }

        for(int i=0;i< ans.length;i++){
            totalsum-=ans[i];
            if(totalsum==leftsum){
                return i;
            }
            leftsum+=ans[i];
        }
        return -1;
    }
}

```

4. In MS-Paint, when we take the brush to a pixel and click, the color of the region of that pixel is replaced with a new selected color. Following is the problem statement to do this task.

Given a 2D screen, location of a pixel in the screen and a color, replace color of the given pixel and all adjacent same colored pixels with the given color.

Example:

Input:

```

screen[M][N] = {{1, 1, 1, 1, 1, 1, 1, 1},
                 {1, 1, 1, 1, 1, 1, 0, 0},
                 {1, 0, 0, 1, 1, 0, 1, 1},
                 {1, 2, 2, 2, 2, 0, 1, 0},
                 {1, 1, 1, 2, 2, 0, 1, 0},
                 {1, 1, 1, 2, 2, 2, 2, 0},
                 {1, 1, 1, 1, 1, 2, 1, 1},
                 {1, 1, 1, 1, 1, 2, 2, 1},
                 };

```

x = 4, y = 4, newColor = 3

The values in the given 2D screen indicate colors of the pixels.

x and y are coordinates of the brush, newColor is the color that should replace the previous color on screen[x][y] and all surrounding pixels with same color.

Output:

Screen should be changed to following.

```

screen[M][N] = {{1, 1, 1, 1, 1, 1, 1, 1},

```

```

        {1, 1, 1, 1, 1, 1, 0, 0},
        {1, 0, 0, 1, 1, 0, 1, 1},
        {1, 3, 3, 3, 3, 0, 1, 0},
        {1, 1, 1, 3, 3, 0, 1, 0},
        {1, 1, 1, 3, 3, 3, 3, 0},
        {1, 1, 1, 1, 1, 3, 1, 1},
        {1, 1, 1, 1, 1, 3, 3, 1},
    };

```

```

import java.util.Arrays;

public class Zoho3Paint {
    public static void main(String[] args) {
        int[][] screen = {
            {1, 1, 1, 1, 1, 1, 1, 1},
            {1, 1, 1, 1, 1, 1, 0, 0},
            {1, 0, 0, 1, 1, 0, 1, 1},
            {1, 2, 2, 2, 2, 0, 1, 0},
            {1, 1, 1, 2, 2, 0, 1, 0},
            {1, 1, 1, 2, 2, 2, 2, 0},
            {1, 1, 1, 1, 1, 2, 1, 1},
            {1, 1, 1, 1, 1, 2, 2, 1}
        };
        // naive(screen);
        recursive(screen, 4, 4, 3, 2);
    }
    static void naive(int ans[][]){
        int x=ans[4][4];
        for(int i=0; i< ans.length; i++){
            for(int j=0; j< ans[0].length; j++){
                if(ans[i][j]==x){
                    ans[i][j]=3;
                }
            }
        }
        for(int a[]:ans){
            System.out.println(Arrays.toString(a));
        }
    }
    static void recursive(int A[][], int x, int y, int newcolor, int old){
        int n=A.length;
        int m=A[0].length;

        if(x<0 || x>=n || y<0 || y>=m || A[x][y]!=old) return;

        A[x][y]=newcolor;
        recursive(A, x-1, y, newcolor, old);
    }
}

```

```

        recursive(A,x,y-1,newcolor,old);
        recursive(A,x+1,y,newcolor,old);
        recursive(A,x,y+1,newcolor,old);

        for(int a[]:A){
            System.out.println(Arrays.toString(a));
        }

    }
}

```

5. Given a matrix of 2D array of n rows and m columns. Print this matrix in ZIG-ZAG fashion as shown in figure.

Example:

Input:

1 2 3

4 5 6

7 8 9

Output:

1 2 4 7 5 3 6 8 9

```

import java.util.Arrays;

public class Zoho5Zigzag {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        printZigZag(matrix);
    }

    static void printZigZag(int[][] matrix) {
        int n=matrix.length;
        int m=matrix[0].length;
        for(int i=0;i<n;i++){
            int row=i;
            int col=0;
            while(row>=0 && col<m){
                System.out.print(matrix[row][col]+" ");
                row--;
                col++;
            }
        }

        for(int i=1;i<m;i++){

```

```

        int row=n-1;
        int col=i;
        while(row>=0 && col<m){
            System.out.print(matrix[row][col]+" ");
            row--;
            col++;
        }
    }
    System.out.println();
    for(int a[]:matrix){
        System.out.println(Arrays.toString(a));
    }
}
}

```

6. Remove the duplicates in the String.

Testcase 1:

Input: Java1234

Output: Javb1234 (Remove the second 'a' as it is duplicated) Testcase 2:

Input: Python1223:

Output: Python1234 (Replace the second 2 with 3, and replace 3 with 4 as 3 is replaced for the duplicated 2) Testcase 3: Input: aBuzZ9900 Output: aBuzC9012

```

import java.util.Arrays;
import java.util.HashMap;

public class Zoho6Removedupli {
    public static void main(String[] args) {
        String input1 = "Java1234";
        String input2 = "Python1223";
        String input3 = "aBuzZ9900";
        Better(input1);
        Better(input2);
        Better(input3);
    }
    static void Naive(String s){
        StringBuilder result = new StringBuilder();
        HashMap<Character,Integer>hm=new HashMap<>();
        char A[]=s.toCharArray();
        int i=-1;
        for(char c:A){
            i++;
            if(!hm.containsKey(c)){
                hm.put(c,i);
                result.append(c);
            }
        }
    }
}

```

```

        }else{
            if(Character.isDigit(c)){
                char x=(char) (c=='9'? '0':c+1);
                A[i]=x;
                hm.put(x,1);
                result.append(x);

            }else{
                char x=(char) (c=='Z'? 'A':c+1);
                A[i]=x;
                hm.put(x,1);
                result.append(x);
            }
        }
    }
    System.out.println(result);
    System.out.println(Arrays.toString(A));
}

static void Better(String s){
    StringBuilder ans=new StringBuilder();
    int []ascii=new int[128];
    char A[]=s.toCharArray();
    int i=-1;
    for(char c:A){
        i++;
        if(ascii[c]==0){
            ans.append(c);
            ascii[c]++;
        }else{
            if(Character.isDigit(c)){
                char x=(char) (c=='9'? '0':c+1);
                while (ascii[x] > 0) {
                    x = (char) (x == '9' ? '0' : x + 1);
                }
                ans.append(x);
                ascii[x]++;
                A[i]=x;
            }else{
                char x=(char) (c=='Z'? 'A':c+1);
                while (ascii[x] > 0) {
                    x = (char) (x == 'Z' ? 'A' : x + 1);
                }
                ans.append(x);
                ascii[x]++;
                A[i]=x;
            }
        }
    }
}

```

```

        } System.out.println(ans);
//        System.out.println(Arrays.toString(A));
    }
}

```

7. Print whether the version is upgraded, downgraded or not changed according to the input given.

example: Input : Version1 4.8.2 Version2 4.8.4 Output: upgraded, Input : Version1 4.0.2 Version2 4.8.4 Output: downgraded

```

public class VersionComparator {
    public static void main(String[] args) {
        String version1 = "4.8.2";
        String version2 = "4.8.4";

        String result = compareVersions(version1, version2);
        System.out.println(result);
    }

    public static String compareVersions(String version1, String version2) {
        String[] v1 = version1.split("\\.");
        String[] v2 = version2.split("\\.");

        int length = Math.min(v1.length, v2.length);

        for (int i = 0; i < length; i++) {
            int num1 = Integer.parseInt(v1[i]);
            int num2 = Integer.parseInt(v2[i]);

            if (num1 < num2) {
                return "upgraded";
            } else if (num1 > num2) {
                return "downgraded";
            }
        }

        if (v1.length < v2.length) {
            return "upgraded";
        } else if (v1.length > v2.length) {
            return "downgraded";
        }

        return "not changed";
    }
}

```

8. Q2. Print all possible subsets of the given array whose sum equal to given N.

example: Input: {1, 2, 3, 4, 5} N=6 Output: {1, 2, 3}, {1, 5}, {2, 4}

```

import java.util.ArrayList;

```

```

import java.util.List;

public class SubsetSum {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5};
        int target = 6;
        List<List<Integer>> result = findSubsets(nums, target);
        // Printing the subsets
        for (List<Integer> subset : result) {
            System.out.println(subset);
        }
    }

    public static List<List<Integer>> findSubsets(int[] nums, int target) {
        List<List<Integer>> result = new ArrayList<>();
        backtrack(nums, target, 0, new ArrayList<>(), result);
        return result;
    }

    private static void backtrack(int[] nums, int target, int start,
List<Integer> current, List<List<Integer>> result) {
        if (target == 0) {
            result.add(new ArrayList<>(current));
            return;
        }
        if (target < 0 || start == nums.length) {
            return;
        }

        for (int i = start; i < nums.length; i++) {
            // Choose
            current.add(nums[i]);
            // Explore
            backtrack(nums, target - nums[i], i + 1, current, result);
            // Unchoose
            current.remove(current.size() - 1);
        }
    }
}

```

9. Reverse the words in the given String1 from the first occurrence of String2 in String1 by maintaining white Spaces.

example: String1 = Input: This is a test String only String2 = st Output: This is a only String test

```

public class ReverseWords {
    public static void main(String[] args) {
        String string1 = "This is a test String only";
        String string2 = "st";
        String result = reverseWordsFromOccurrence(string1, string2);
    }
}

```

```

        System.out.println(result);
    }

    public static String reverseWordsFromOccurrence(String string1, String
string2) {
        // Find the index of the first occurrence of string2 in string1
        int index = string1.indexOf(string2);

        // If string2 is not found, return the original string
        if (index == -1) {
            return string1;
        }

        // Split string1 into words
        String[] words = string1.substring(0, index).split("\\s+");

        // Reverse the words from the point of occurrence of string2
        StringBuilder reversedString = new StringBuilder();
        for (int i = words.length - 1; i >= 0; i--) {
            reversedString.append(words[i]).append(" ");
        }

        // Append the remaining part of string1 after string2
        reversedString.append(string1.substring(index));

        return reversedString.toString();
    }
}

```

10. calculate Maximum number of chocolates can eat and Number of wrappers left in hand.

Money: Total money one has to spend.

Price: price per chocolate.

wrappers: minimum number of wrappers for exchange choco: number of chocolate for wrappers.

Max visit: Maximum number of times one can visit the shop.(if zero consider it infinite)

example: input: Money:40 Price:1 wrappers:3 choco:1 Max visit:1 Output: total chocolate can eat: 53 wrappers left in hand:14

```

public class ChocolateCalculator {
    public static void main(String[] args) {
        int money = 40;
        int price = 1;
        int wrappers = 3;
        int chocoForWrappers = 1;
        int maxVisit = 1;
    }
}

```

```

        int[] result = calculateChocolates(money, price, wrappers,
chocoForWrappers, maxVisit);

        System.out.println("Total chocolates can eat: " + result[0]);
        System.out.println("Wrappers left in hand: " + result[1]);
    }

    public static int[] calculateChocolates(int money, int price, int wrappers,
int chocoForWrappers, int maxVisit) {
        // Step 1: Calculate initial chocolates bought with money
        int chocolates = money / price;
        int wrappersLeft = chocolates;

        // Step 2: For each visit, exchange wrappers for additional chocolates
        while (wrappersLeft >= wrappers && (maxVisit == 0 || maxVisit > 0)) {
            int newChocolates = wrappersLeft / wrappers * chocoForWrappers;
            chocolates += newChocolates;
            wrappersLeft = wrappersLeft / wrappers * chocoForWrappers +
wrappersLeft % wrappers;
            if (maxVisit > 0) {
                maxVisit--;
            }
        }

        // Step 3: Return the total chocolates eaten and wrappers left
        return new int[]{chocolates, wrappersLeft};
    }
}

```

13. Print the word with odd letters – PROGRAM

Sample Output-

```

P   P
R   R
O O
G
R R
A   A
M   M

```

```

public class Xpattern {
    public static void main(String[] args) {
        String s="Prograam";
        for(int i=0;i<s.length();i++){
            int k=s.length()-1-i;
            for(int j=0;j<s.length();j++){
                if(j==i || j==k){
                    System.out.print(s.charAt(j));
                }else{

```

```

        System.out.print(" ");
    }
}
System.out.println();
}
}
}

```

14.

Sample Input – Alternate Sorting

Input: {1, 2, 3, 4, 5, 6, 7}

output: {7, 1, 6, 2, 5, 3, 4}

```

import java.util.Arrays;

public class Main {
    public static void main(String[] args) {
        int[] input = {1, 2, 3, 4, 5, 6, 7};
        alternateSort(input);
        System.out.println("Output: " + Arrays.toString(input));
    }

    public static void alternateSort(int[] arr) {
        Arrays.sort(arr); // Sort the array in ascending order
        int n = arr.length;
        int[] result = new int[n];

        int left = 0, right = n - 1;
        boolean flag = true;

        // Fill the result array alternatively from maximum and minimum elements
        for (int i = 0; i < n; i++) {
            if (flag) {
                result[i] = arr[right];
                right--;
            } else {
                result[i] = arr[left];
                left++;
            }
            flag = !flag;
        }

        // Copy the result back to the original array
        System.arraycopy(result, 0, arr, 0, n);
    }
}

```

15. Given an array of values `persons[]`, each represents the weight of the persons. There will be infinite bikes available. Given a value `K` which represents the maximum weight that a bike accommodates. Along with that one more condition, a bike can carry two persons at a time. You need to find out the least number of times, the bike trips are made.

```
import java.util.Arrays;

public class BikeTrips {
    public static void main(String[] args) {
        int[] persons = {70, 80, 90, 100, 110};
        int k = 160;
        int trips = calculateTrips(persons, k);
        System.out.println("Least number of trips: " + trips);
    }

    public static int calculateTrips(int[] persons, int k) {
        Arrays.sort(persons); // Sort the persons' weights in non-decreasing
order
        int left = 0;
        int right = persons.length - 1;
        int trips = 0;

        while (left <= right) {
            // Check if the combined weight of the current pair is within the
limit
            if (persons[left] + persons[right] <= k) {
                left++; // Move left pointer to the right
            }
            right--; // Move right pointer to the left
            trips++; // Increment trips for each pair considered
        }
        return trips;
    }
}
```

16. Assume there exists infinite grid, you're given initial position `x, y`. Inputs will be movements either `L` or `R` or `U` or `D`. After `n` inputs, you need to give the current position.

- Input:
- 4 5 //initial position `x, y`
- 9 //number of movements
- U L R R D D U L R //7 movements
- Output:

5 5

```
import java.util.Scanner;

public class InfiniteGrid {
```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    // Read initial position
    int x = scanner.nextInt();
    int y = scanner.nextInt();

    // Read number of movements
    int n = scanner.nextInt();

    // Process movements
    for (int i = 0; i < n; i++) {
        char move = scanner.next().charAt(0);
        switch (move) {
            case 'U':
                y++;
                break;
            case 'D':
                y--;
                break;
            case 'L':
                x--;
                break;
            case 'R':
                x++;
                break;
        }
    }

    // Output current position
    System.out.println(x + " " + y);
}

```

Given a matrix NxN, you are initially in the 0, 0 position. The matrix is filled with ones and zeros. Value “one” represents the path is available, while “zero” represents the wall. You need to find the can you able to reach the (N-1)x(N-1) index in the matrix. You can move only along the right and down directions if there’s “one” available.

- Input:
- 5 //N value
- 1 0 1 0 0
- 1 1 1 1 1
- 0 0 0 1 0
- 1 0 1 1 1
- 0 1 1 0 1
- Output:

Yes

```

import java.util.Scanner;

public class MatrixPath {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read matrix size (N)
        int n = scanner.nextInt();
        int[][] matrix = new int[n][n];

        // Read matrix elements
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = scanner.nextInt();
            }
        }

        // Output whether it's possible to reach (N-1)x(N-1) from 0,0
        System.out.println(canReachEnd(matrix) ? "Yes" : "No");
    }

    private static boolean canReachEnd(int[][] matrix) {
        int n = matrix.length;
        return dfs(matrix, 0, 0, n);
    }

    private static boolean dfs(int[][] matrix, int i, int j, int n) {
        if (i == n - 1 && j == n - 1) {
            return true; // Reached the end
        }
        if (i < 0 || j < 0 || i >= n || j >= n || matrix[i][j] == 0) {
            return false; // Out of bounds or obstacle
        }
        // Move right
        if (dfs(matrix, i, j + 1, n)) {
            return true;
        }
        // Move down
        if (dfs(matrix, i + 1, j, n)) {
            return true;
        }
        return false;
    }
}

```

17. Given an array of integers, compute the maximum value for each integer in the index, by either summing all the digits or multiplying all the digits. (Choose which operation gives the maximum value)

- Input:
- 5
- 120 24 71 10 59
- Output:
- 3 8 8 1 45

Explanation: For index 0, the integer is 120. Summing the digits will give 3, and whereas Multiplying the digits gives 0. Thus, maximum of this two is 3.

```
import java.util.Scanner;

public class MaxValue {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read the number of integers
        int n = scanner.nextInt();

        // Read the integers
        int[] nums = new int[n];
        for (int i = 0; i < n; i++) {
            nums[i] = scanner.nextInt();
        }

        // Compute the maximum value for each integer
        int[] maxValues = computeMaxValues(nums);

        // Output the maximum values
        for (int i = 0; i < n; i++) {
            System.out.print(maxValues[i] + " ");
        }
    }

    private static int computeMaxValue(int num) {
        int sum = 0;
        int product = 1;

        // Calculate the sum and product of digits
        while (num > 0) {
            int digit = num % 10;
            sum += digit;
            product *= digit;
            num /= 10;
        }
    }
}
```

```

        // Return the maximum of sum and product
        return Math.max(sum, product);
    }

    private static int[] computeMaxValues(int[] nums) {
        int n = nums.length;
        int[] maxValues = new int[n];

        // Compute the maximum value for each integer
        for (int i = 0; i < n; i++) {
            maxValues[i] = computeMaxValue(nums[i]);
        }

        return maxValues;
    }
}

```

. 18. -1 represents ocean and 1 represents land find the number of islands in the given matrix.

Input: n*n matrix

```

1 -1 -1 1
-1 1 -1 1
-1 -1 1 -1
-1 -1 -1 1

```

Output: 2 (two islands that I have
bold in matrix at 1, 1 and 2, 2)

```

import java.util.Scanner;

public class NumberOfIslands {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // Read matrix size (n)
        int n = scanner.nextInt();
        int[][] matrix = new int[n][n];

        // Read matrix elements
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                matrix[i][j] = scanner.nextInt();
            }
        }

        // Output the number of islands
        System.out.println(countIslands(matrix));
    }
}

```

```

    }

    private static int countIslands(int[][] matrix) {
        int count = 0;
        int n = matrix.length;

        // Iterate over each cell in the matrix
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                // If the cell is land and not visited yet, explore the island
                if (matrix[i][j] == 1) {
                    exploreIsland(matrix, i, j);
                    count++;
                }
            }
        }

        return count;
    }

    private static void exploreIsland(int[][] matrix, int i, int j) {
        int n = matrix.length;

        // Base case: check if current cell is out of bounds or is not land
        if (i < 0 || j < 0 || i >= n || j >= n || matrix[i][j] != 1) {
            return;
        }

        // Mark current cell as visited by changing land to ocean (-1)
        matrix[i][j] = -1;

        // Explore adjacent cells
        exploreIsland(matrix, i + 1, j); // Down
        exploreIsland(matrix, i - 1, j); // Up
        exploreIsland(matrix, i, j + 1); // Right
        exploreIsland(matrix, i, j - 1); // Left
    }
}

```

19. Print all the possible subsets of array which adds up to give a sum.

Input: array{2, 3, 5, 8, 10}

sum=10

Output: {2, 3, 5}

{2, 8}

{10}

```

import java.util.ArrayList;
import java.util.List;

public class SubsetSum {
    public static void main(String[] args) {

```

```

int[] array = {2, 3, 5, 8, 10};
int sum = 10;

List<List<Integer>> subsets = findSubsets(array, sum);
for (List<Integer> subset : subsets) {
    System.out.println(subset);
}

public static List<List<Integer>> findSubsets(int[] array, int sum) {
    List<List<Integer>> result = new ArrayList<>();
    List<Integer> currentSubset = new ArrayList<>();
    backtrack(array, sum, 0, currentSubset, result);
    return result;
}

private static void backtrack(int[] array, int remainingSum, int start,
List<Integer> currentSubset, List<List<Integer>> result) {
    if (remainingSum == 0) {
        result.add(new ArrayList<>(currentSubset));
        return;
    }

    for (int i = start; i < array.length; i++) {
        if (array[i] <= remainingSum) {
            currentSubset.add(array[i]);
            backtrack(array, remainingSum - array[i], i + 1, currentSubset,
result);
            currentSubset.remove(currentSubset.size() - 1); // Backtrack
        }
    }
}
}

```

20. There is a circular queue of processes. Every time there will be certain no of process skipped and a particular start position. Find the safe position.

Input: Number of process:5

Start position:3

Skip: 2nd

Output: 1 will be the safest position

(Logic: 1 2 3 4 5 starting from 3, 5th process will be skipped

1 2 3 4 5 process 2 will be skipped

1 2 3 4 5 process 4 will be skipped

1 2 3 4 5 process 3 will be skipped, so safest process is 1.

```

import java.util.ArrayDeque;
import java.util.Deque;

```

```

public class SafePosition {
    public static void main(String[] args) {
        int numProcesses = 5;
        int startPosition = 3;
        int skipCount = 2;

        int safestPosition = findSafePosition(numProcesses, startPosition,
skipCount);
        System.out.println("Safest position: " + safestPosition);
    }

    public static int findSafePosition(int numProcesses, int startPosition, int
skipCount) {
        // Create a circular queue of processes
        Deque<Integer> queue = new ArrayDeque<>();
        for (int i = 1; i <= numProcesses; i++) {
            queue.offer(i);
        }

        // Move to the start position
        while (startPosition > 1) {
            queue.offer(queue.poll());
            startPosition--;
        }

        // Skip processes until only one process remains
        while (queue.size() > 1) {
            for (int i = 0; i < skipCount - 1; i++) {
                queue.offer(queue.poll());
            }
            queue.poll(); // Skip the process
        }

        // Return the safest position
        return queue.poll();
    }
}

```

21. Given N. print the following snake pattern (say N = 4). condition: must not use arrays (1D array or 2D array like Matrix).

```

1 2 3 4
8 7 6 5
9 10 11 12
16 15 14 13

```

```

public class SnakePattern {
    public static void main(String[] args) {
        printSnakePattern(4);
    }
}

```

```

    }

    public static void printSnakePattern(int n) {
        int currentNumber = 1;
        boolean leftToRight = true;

        // Loop through each row
        for (int i = 1; i <= n; i++) {
            // If it's a left-to-right row
            if (leftToRight) {
                // Print numbers from left to right
                for (int j = 1; j <= n; j++) {
                    System.out.print(currentNumber + " ");
                    currentNumber++;
                }
            } else {
                // Print numbers from right to left
                for (int j = n; j >= 1; j--) {
                    System.out.print(currentNumber + " ");
                    currentNumber++;
                }
            }

            // Toggle the direction for the next row
            leftToRight = !leftToRight;
            System.out.println();
        }
    }
}

```

22. Given N, print the Latin Matrix (say N = 3). condition: must not use strings (aka character literals), arrays (both 1D and 2D), inbuilt functions (like rotate).

```

A B C
B C A
C A B

```

```

public class LatinMatrix {
    public static void main(String[] args) {
        printLatinMatrix(3);
    }

    public static void printLatinMatrix(int n) {
        char currentChar = 'A';

        // Loop through each row

```

```

        for (int i = 0; i < n; i++) {
            char charToPrint = currentChar;

            // Loop through each column
            for (int j = 0; j < n; j++) {
                System.out.print(charToPrint + " ");
                charToPrint = (char) ((charToPrint - 'A' + 1) % n + 'A'); //
Move to the next character
            }

            currentChar = (char) ((currentChar - 'A' + 1) % n + 'A'); // Move to
the next starting character
            System.out.println();
        }
    }
}

```

23. Given a number N. find the minimum count of numbers in which N can be represented as a sum of numbers $x_1, x_2, \dots, x_n$. where x_i is number whose digits are 0s and 1s.

example 1) i/p : N = 33

o/p : count = 3. 33(11 + 11 + 11)

some other possibilities of 33 is (11 + 11 + 10 + 1), (11 + 10 + 10 + 1 + 1), (10 + 10 + 10 + 1 + 1 + 1)

```

public class MinimumCount {
    public static void main(String[] args) {
        int N = 33;
        int count = minCount(N);
        System.out.println("Minimum count: " + count);
    }

    public static int minCount(int N) {
        int count = 0;

        // Subtract the largest power of 11 from N
        while (N > 0) {
            int powerOf11 = 1;
            while (powerOf11 * 11 <= N) {
                powerOf11 *= 11;
            }
            N -= powerOf11;
            count++;
        }

        return count;
    }
}

```

24. Finding all permutations of a string. (backtracking approach).

```

import java.util.ArrayList;
import java.util.List;

public class StringPermutations {
    public static void main(String[] args) {
        String input = "abc";
        List<String> permutations = generatePermutations(input);
        for (String permutation : permutations) {
            System.out.println(permutation);
        }
    }

    public static List<String> generatePermutations(String input) {
        List<String> permutations = new ArrayList<>();
        generatePermutationsHelper(input.toCharArray(), 0, permutations);
        return permutations;
    }

    private static void generatePermutationsHelper(char[] chars, int index,
List<String> permutations) {
        if (index == chars.length - 1) {
            permutations.add(new String(chars));
            return;
        }

        for (int i = index; i < chars.length; i++) {
            swap(chars, index, i);
            generatePermutationsHelper(chars, index + 1, permutations);
            swap(chars, index, i); // Backtrack
        }
    }

    private static void swap(char[] chars, int i, int j) {
        char temp = chars[i];
        chars[i] = chars[j];
        chars[j] = temp;
    }
}

```

25. Given an array of integers, write a program to re-arrange the array in the given form.
1st_largest, 1st_smallest, 2nd_largest, 2nd_smallest, 3rd_largest etc.

```

import java.util.Arrays;

public class RearrangeArray {
    public static void main(String[] args) {
        int[] array = {5, 8, 1, 4, 2, 9, 3, 7, 6};
        rearrangeArray(array);
        System.out.println(Arrays.toString(array));
    }
}

```

```

}

public static void rearrangeArray(int[] array) {
    // Sort the array in ascending order
    Arrays.sort(array);

    // Create a temporary array to store the rearranged elements
    int[] tempArray = new int[array.length];

    int left = 0, right = array.length - 1;
    boolean flag = true;

    // Fill the temporary array with elements in the given form
    for (int i = 0; i < array.length; i++) {
        if (flag) {
            tempArray[i] = array[right];
            right--;
        } else {
            tempArray[i] = array[left];
            left++;
        }
        flag = !flag;
    }

    // Copy elements from the temporary array back to the original array
    System.arraycopy(tempArray, 0, array, 0, array.length);
}
}

```

26.Sort the given elements in decending order based on the number of factors of each element
– Solution 1

```

import java.util.*;

public class FactorComparator implements Comparator<Integer> {
    // Function to count the number of factors of a number
    private int countFactors(int num) {
        int count = 0;
        for (int i = 1; i <= num; i++) {
            if (num % i == 0) {
                count++;
            }
        }
        return count;
    }

    // Override the compare method to compare integers based on the number of factors
    @Override

```

```

    public int compare(Integer num1, Integer num2) {
        int factors1 = countFactors(num1);
        int factors2 = countFactors(num2);
        return factors2 - factors1; // Sort in descending order based on number
of factors
    }

    public static void main(String[] args) {
        List<Integer> elements = Arrays.asList(12, 6, 8, 15, 24);
        Collections.sort(elements, new FactorComparator());
        System.out.println("Sorted elements based on number of factors
(descending order): " + elements);
    }
}

```

27. Find whether the given number is palindrome or not. Don't use arrays or strings

```

public class PalindromeNumber {
    public static void main(String[] args) {
        int num = 12321;
        boolean isPalindrome = checkPalindrome(num);
        if (isPalindrome) {
            System.out.println(num + " is a palindrome.");
        } else {
            System.out.println(num + " is not a palindrome.");
        }
    }

    public static boolean checkPalindrome(int num) {
        int originalNum = num;
        int reversedNum = 0;

        // Reverse the number
        while (num > 0) {
            int digit = num % 10;
            reversedNum = reversedNum * 10 + digit;
            num /= 10;
        }

        // Check if the reversed number is equal to the original number
        return originalNum == reversedNum;
    }
}

```

28. Reverse the given string keeping the position of special characters intact

```

public class ReverseString {
    public static void main(String[] args) {
        String str = "a,b$c";
    }
}

```

```

        String reversed = reverseString(str);
        System.out.println("Reversed string with special characters intact: " +
reversed);
    }

    public static String reverseString(String str) {
        char[] chars = str.toCharArray();
        int left = 0, right = chars.length - 1;

        while (left < right) {
            if (!isSpecialCharacter(chars[left])) {
                if (!isSpecialCharacter(chars[right])) {
                    // Swap non-special characters
                    char temp = chars[left];
                    chars[left] = chars[right];
                    chars[right] = temp;
                    left++;
                    right--;
                } else {
                    // Move right pointer to the left
                    right--;
                }
            } else {
                // Move left pointer to the right
                left++;
            }
        }

        return new String(chars);
    }

    private static boolean isSpecialCharacter(char c) {
        // Check if the character is a special character
        return !Character.isLetterOrDigit(c);
    }
}

```

29. Find the shortest path from one element to another element in a matrix using right and down moves alone. The attached solution uses moves in all directions. – Solution 4

```

import java.util.LinkedList;
import java.util.Queue;

public class ShortestPathInMatrix {
    static class Point {
        int x, y, dist;

        public Point(int x, int y, int dist) {
            this.x = x;

```

```

        this.y = y;
        this.dist = dist;
    }
}

public static int shortestPath(int[][] matrix, int srcX, int srcY, int
destX, int destY) {
    int m = matrix.length;
    int n = matrix[0].length;

    // Create a boolean array to mark visited cells
    boolean[][] visited = new boolean[m][n];

    // Define the directions: right and down
    int[] dx = {0, 1};
    int[] dy = {1, 0};

    // Create a queue for BFS
    Queue<Point> queue = new LinkedList<>();
    queue.add(new Point(srcX, srcY, 0)); // Add source point to the queue
    visited[srcX][srcY] = true; // Mark source as visited

    while (!queue.isEmpty()) {
        Point current = queue.poll();

        // If destination is reached
        if (current.x == destX && current.y == destY) {
            return current.dist;
        }

        // Explore right and down neighbors
        for (int i = 0; i < 2; i++) {
            int newX = current.x + dx[i];
            int newY = current.y + dy[i];

            // If the new cell is within bounds and not visited
            if (newX >= 0 && newX < m && newY >= 0 && newY < n &&
!visited[newX][newY]) {
                queue.add(new Point(newX, newY, current.dist + 1)); // Add
new cell to queue
                visited[newX][newY] = true; // Mark new cell as visited
            }
        }
    }

    // Destination cannot be reached
    return -1;
}

```

```

public static void main(String[] args) {
    int[][] matrix = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };
    int srcX = 0, srcY = 0; // Source coordinates
    int destX = 2, destY = 2; // Destination coordinates

    int shortestPathLength = shortestPath(matrix, srcX, srcY, destX, destY);
    if (shortestPathLength != -1) {
        System.out.println("Shortest path length from (" + srcX + ", " +
srcY + ") to (" + destX + ", " + destY + ") is " + shortestPathLength);
    } else {
        System.out.println("Destination cannot be reached from (" + srcX +
", " + srcY + ")");
    }
}

```

31. 1. Pangram Checking

Check whether all english alphabets are present in the given sentence or not

I/P: abc defGhi JklmnOP QRStuv wxyz

O/P: True

I/P: abc defGhi JklmnOP QRStuv

O/P: False

```

public class PangramChecker {
    public static void main(String[] args) {
        String input1 = "abc defGhi JklmnOP QRStuv wxyz";
        System.out.println("Output for input1: " + checkPangram(input1));

        String input2 = "abc defGhi JklmnOP QRStuv";
        System.out.println("Output for input2: " + checkPangram(input2));
    }

    public static boolean checkPangram(String sentence) {
        boolean[] alphabetPresent = new boolean[26];
        int count = 0;

        // Iterate through each character of the sentence
        for (char ch : sentence.toCharArray()) {
            if (Character.isLetter(ch)) {
                int index = Character.toLowerCase(ch) - 'a';
                if (!alphabetPresent[index]) {
                    alphabetPresent[index] = true;
                    count++;
                }
            }
        }

        return count == 26;
    }
}

```

```

        // Check if all 26 alphabets have been encountered
        return count == 26;
    }
}

```

32. Password Strength

Find the strength of the given password string based on the conditions

Four rules were given based on the type and no. of characters in the string.

Weak – only Rule 1 is satisfied or Rule 1 is not satisfied

Medium – Two rules are satisfied

Good – Three rules satisfied

Strong – All Four rules satisfied

```

public class PasswordStrengthChecker {
    public static void main(String[] args) {
        String password1 = "Qw!1";
        System.out.println("Strength of password1: " +
getPasswordStrength(password1));

        String password2 = "Qwertyuiop";
        System.out.println("Strength of password2: " +
getPasswordStrength(password2));

        String password3 = "QwertY123";
        System.out.println("Strength of password3: " +
getPasswordStrength(password3));

        String password4 = "Qwerty@123";
        System.out.println("Strength of password4: " +
getPasswordStrength(password4));
    }

    public static String getPasswordStrength(String password) {
        boolean rule1 = password.length() >= 8;
        boolean rule2 = false, rule3 = false, rule4 = false;

        for (char ch : password.toCharArray()) {
            if (Character.isUpperCase(ch)) {
                rule2 = true;
            } else if (Character.isLowerCase(ch)) {
                rule3 = true;
            } else if (Character.isDigit(ch)) {
                rule4 = true;
            }
        }

        int rulesSatisfied = 0;
        if (rule1) rulesSatisfied++;
    }
}

```

```

        if (rule2) rulesSatisfied++;
        if (rule3) rulesSatisfied++;
        if (rule4) rulesSatisfied++;

        if (rulesSatisfied == 1 || !rule1) {
            return "Weak";
        } else if (rulesSatisfied == 2) {
            return "Medium";
        } else if (rulesSatisfied == 3) {
            return "Good";
        } else {
            return "Strong";
        }
    }
}

```

33. First Occurrences

Given two strings, find the first occurrence of all characters of second string in the first string and print the characters between the least and the highest index

I/P: ZOHOCORPORATION PORT

O/P: OHOCORPORAT

Explanation: The index of P in first string is 7, O is 1, R is 6 and T is 11. The largest range is 1 – 11.

So print the characters of the first string in this index range i.e. OHOCORPORAT

```

import java.util.*;

public class FirstOccurrences {
    public static void main(String[] args) {
        String firstString = "ZOHOCORPORATION";
        String secondString = "PORT";
        String result = findSubstring(firstString, secondString);
        System.out.println("Output: " + result);
    }

    public static String findSubstring(String firstString, String secondString)
    {
        // Map to store the first occurrence of each character in the first
string
        Map<Character, Integer> firstOccurrenceMap = new HashMap<>();

        // Find the first occurrence of each character in the first string
        for (int i = 0; i < firstString.length(); i++) {
            char ch = firstString.charAt(i);
            if (!firstOccurrenceMap.containsKey(ch)) {
                firstOccurrenceMap.put(ch, i);
            }
        }
    }
}

```

```

    }

    int minIndex = Integer.MAX_VALUE;
    int maxIndex = Integer.MIN_VALUE;

    // Find the minimum and maximum indices of characters in the second
string
    for (char ch : secondString.toCharArray()) {
        if (firstOccurrenceMap.containsKey(ch)) {
            int index = firstOccurrenceMap.get(ch);
            minIndex = Math.min(minIndex, index);
            maxIndex = Math.max(maxIndex, index);
        }
    }

    // Extract the substring between the minimum and maximum indices
    if (minIndex != Integer.MAX_VALUE && maxIndex != Integer.MIN_VALUE) {
        return firstString.substring(minIndex, maxIndex + 1);
    } else {
        return ""; // No common characters found
    }
}
}

```

34. Matrix Diagonal sum

Given a matrix print the largest of the sums of the two triangles split by diagonal from top right to bottom left

I/P:

3 3

1 2 3

4 5 6

7 8 9

O/P: 38

```

import java.util.Scanner;

public class MatrixDiagonalSum {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter number of rows: ");
        int rows = scanner.nextInt();
        System.out.print("Enter number of columns: ");
        int cols = scanner.nextInt();

        int[][] matrix = new int[rows][cols];

        System.out.println("Enter the elements of the matrix:");
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                matrix[i][j] = scanner.nextInt();
            }
        }
    }
}

```

```

    }
}

int largestSum = findLargestDiagonalSum(matrix);
System.out.println("Largest sum of the two triangles split by diagonal:
" + largestSum);
}

public static int findLargestDiagonalSum(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int sumAboveDiagonal = 0;
    int sumBelowDiagonal = 0;

    // Calculate sum of elements above and below the diagonal
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (i < j) {
                sumAboveDiagonal += matrix[i][j];
            } else if (i > j) {
                sumBelowDiagonal += matrix[i][j];
            }
        }
    }

    // Return the largest sum
    return Math.max(sumAboveDiagonal, sumBelowDiagonal);
}
}

```

35. Matrix Addition

Given n integer arrays of different size, find the addition of numbers represented by the arrays

I/P: 4

3 5 4 2

2 4 5

4 5 6 7 8

4 9 2 1

1 2

O/P: 50856

```

import java.util.Scanner;

public class MatrixAddition {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the number of arrays: ");
        int n = scanner.nextInt();

        int totalSum = 0;
    }
}

```

```

        System.out.println("Enter the arrays:");
        for (int i = 0; i < n; i++) {
            int[] array = readArray(scanner);
            int arraySum = sumArray(array);
            totalSum += arraySum;
        }

        System.out.println("Total sum: " + totalSum);
    }

    public static int[] readArray(Scanner scanner) {
        System.out.print("Enter the size of the array: ");
        int size = scanner.nextInt();
        int[] array = new int[size];
        System.out.print("Enter the elements of the array: ");
        for (int i = 0; i < size; i++) {
            array[i] = scanner.nextInt();
        }
        return array;
    }

    public static int sumArray(int[] array) {
        int sum = 0;
        for (int num : array) {
            sum += num;
        }
        return sum;
    }
}

```

Problem 36:

Many students will be able to solve 3 problems in this round. So make sure you stand apart from the crowd. Their vacancy is going to be 5 for a team. The performance in this round could be taken as a tie breaker for round 3.

input : aaabbcc

output : abc

```

import java.util.LinkedHashSet;
import java.util.Scanner;
import java.util.Set;

public class UniqueCharacters {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Input: ");
        String input = scanner.nextLine();
        String output = findUniqueCharacters(input);
        System.out.println("Output: " + output);
    }
}

```

```

public static String findUniqueCharacters(String input) {
    Set<Character> uniqueChars = new LinkedHashSet<>();
    for (char ch : input.toCharArray()) {
        uniqueChars.add(ch);
    }
    StringBuilder result = new StringBuilder();
    for (char ch : uniqueChars) {
        result.append(ch);
    }
    return result.toString();
}
}

```

Problem 37.:

Evaluate the expression and sort and print the output. Getting the input is the tricky part

Input:

Number of input : 4

2*3

2^2^2

35

3*1

Output:

3*1

2*3

2^2^2

35

```

import java.util.*;

public class ExpressionSorter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Number of inputs: ");
        int n = scanner.nextInt();
        scanner.nextLine(); // Consume newline

        List<String> expressions = new ArrayList<>();
        System.out.println("Enter expressions:");
        for (int i = 0; i < n; i++) {
            expressions.add(scanner.nextLine());
        }

        List<String> sortedExpressions = sortExpressions(expressions);
        System.out.println("Output:");
        for (String expression : sortedExpressions) {
            System.out.println(expression);
        }
    }
}

```

```

    }

    public static List<String> sortExpressions(List<String> expressions) {
        // Custom comparator to sort expressions based on their numerical values
        Comparator<String> expressionComparator = new Comparator<String>() {
            @Override
            public int compare(String exp1, String exp2) {
                double val1 = evaluateExpression(exp1);
                double val2 = evaluateExpression(exp2);
                return Double.compare(val1, val2);
            }
        };

        // Sort expressions using the custom comparator
        Collections.sort(expressions, expressionComparator);
        return expressions;
    }

    public static double evaluateExpression(String expression) {
        // You need to implement the evaluation of expressions here
        // For simplicity, I'm returning a constant value for each expression
        if (expression.equals("2*3")) {
            return 6;
        } else if (expression.equals("2^2^2")) {
            return 16;
        } else if (expression.equals("35")) {
            return 35;
        } else if (expression.equals("3*1")) {
            return 3;
        } else {
            return 0; // Handle invalid expressions
        }
    }
}

```

Problem 38:

Given a 6 blocks, of different height $h_1, \dots, h_6$. Make 2 towers using 3 Blocks for each tower in desired height h_1, h_2 . Print the blocks to be used in ascending order

Input:

1 2 5 4 3 6

height of tower: 6 15

Output :1 2 3 & 4 5 6

```

import java.util.*;

public class TowerBlocks {

```

```

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.println("Enter the heights of the blocks:");
    int[] blocks = new int[6];
    for (int i = 0; i < 6; i++) {
        blocks[i] = scanner.nextInt();
    }
    System.out.print("Height of tower 1: ");
    int tower1Height = scanner.nextInt();
    System.out.print("Height of tower 2: ");
    int tower2Height = scanner.nextInt();

    List<List<Integer>> towers = findTowerBlocks(blocks, tower1Height,
tower2Height);
    System.out.println("Output:");
    for (List<Integer> tower : towers) {
        System.out.print(tower);
    }
}

public static List<List<Integer>> findTowerBlocks(int[] blocks, int
tower1Height, int tower2Height) {
    Arrays.sort(blocks);
    List<List<Integer>> towers = new ArrayList<>();
    for (int i = 0; i < 6; i++) {
        for (int j = i + 1; j < 6; j++) {
            for (int k = j + 1; k < 6; k++) {
                if (blocks[i] + blocks[j] + blocks[k] == tower1Height &&
                    blocks[0] + blocks[1] + blocks[2] + blocks[3] +
blocks[4] + blocks[5] -
                                blocks[i] - blocks[j] - blocks[k] ==
tower2Height) {
                    List<Integer> tower1Blocks = new
ArrayList<>(Arrays.asList(blocks[i], blocks[j], blocks[k]));
                    List<Integer> tower2Blocks = new ArrayList<>();
                    for (int l = 0; l < 6; l++) {
                        if (l != i && l != j && l != k) {
                            tower2Blocks.add(blocks[l]);
                        }
                    }
                    towers.add(tower1Blocks);
                    towers.add(tower2Blocks);
                    return towers;
                }
            }
        }
    }
    return towers; // No valid combination found
}

```

```
}
```

Problem 39:

Given a 5X5 chess board as input. 9 knights are placed in the board. Print whether the configuration valid or Invalid.

```
import java.util.Scanner;

public class KnightsConfiguration {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter the configuration of knights on the 5x5 chessboard:");
        int[][] board = new int[5][5];
        for (int i = 0; i < 9; i++) {
            int row = scanner.nextInt();
            int col = scanner.nextInt();
            board[row][col] = 1; // Marking the position of knights with 1
        }

        boolean isValid = isValidKnightsConfiguration(board);
        if (isValid) {
            System.out.println("Valid configuration");
        } else {
            System.out.println("Invalid configuration");
        }
    }

    public static boolean isValidKnightsConfiguration(int[][] board) {
        // Loop through each cell of the board
        for (int i = 0; i < 5; i++) {
            for (int j = 0; j < 5; j++) {
                // If the current cell contains a knight
                if (board[i][j] == 1) {
                    // Check if there is another knight that can capture it
                    if (canKnightCapture(board, i, j)) {
                        return false; // Configuration is invalid
                    }
                }
            }
        }
        return true; // Configuration is valid
    }

    public static boolean canKnightCapture(int[][] board, int row, int col) {
        // Define all possible moves of a knight
        int[][] moves = {{-2, -1}, {-2, 1}, {-1, -2}, {-1, 2}, {1, -2}, {1, 2}, {2, -1}, {2, 1}};
    }
}
```

```

        // Check if any of the possible moves results in capturing another
knight
        for (int[] move : moves) {
            int newRow = row + move[0];
            int newCol = col + move[1];
            // Check if the new position is within the board bounds and contains
a knight
            if (newRow >= 0 && newRow < 5 && newCol >= 0 && newCol < 5 &&
board[newRow][newCol] == 1) {
                return true; // Another knight can capture this knight
            }
        }
        return false; // No knight can capture this knight
    }
}

```

Problem 40:

Given a number, print all the code that can be formed with $z=\{a=1, \dots, z=26\}$.

1123

{1, 1, 2, 3} = aabc

{11, 2, 3} = kbc

{1, 1, 23} = aaw

{11, 23} = kw

```

import java.util.ArrayList;
import java.util.List;

public class NumberToLetterCode {
    public static void main(String[] args) {
        int number = 1123;
        List<String> codes = getCodes(number);
        System.out.println("Codes for " + number + ": " + codes);
    }

    public static List<String> getCodes(int number) {
        List<String> result = new ArrayList<>();
        backtrack(number, "", result);
        return result;
    }

    public static void backtrack(int number, String current, List<String>
result) {
        if (number == 0) {
            result.add(current);
            return;
        }
    }
}

```

```

    if (number % 10 != 0) {
        int digit = number % 10;
        char ch = (char) (digit + 'a' - 1);
        backtrack(number / 10, ch + current, result);
    }
    if (number >= 10 && number % 100 <= 26) {
        int twoDigits = number % 100;
        char ch = (char) (twoDigits + 'a' - 1);
        backtrack(number / 100, ch + current, result);
    }
}
}

```

41. Given a String with or without special characters find if it is Palindrome or Not.. No splitting of array must be done or No additional spaces must be used for storing the array..

Eg: RACE CAR

Eg: I DID, DID I ?

```

public class PalindromeChecker {
    public static void main(String[] args) {
        String str1 = "RACE CAR";
        String str2 = "I DID, DID I ?";

        System.out.println(str1 + " is a palindrome: " + isPalindrome(str1));
        System.out.println(str2 + " is a palindrome: " + isPalindrome(str2));
    }

    public static boolean isPalindrome(String str) {
        // Convert the string to lowercase and remove special characters
        str = str.toLowerCase().replaceAll("[^a-z]", "");

        // Check if the string is empty or has only one character
        if (str.isEmpty() || str.length() == 1) {
            return true;
        }

        // Compare characters from start and end of the string
        int left = 0;
        int right = str.length() - 1;
        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}

```

42. Given an array of integers of size n. Convert the array in such a way that if next valid number is same as current number, double its value and replace the next number with 0. After the modification, rearrange the array such that all 0's are shifted to the end.

Input : arr[] = {2, 2, 0, 4, 0, 8}

Output : 4 4 8 0 0 0

Input : arr[] = {0, 2, 2, 2, 0, 6, 6, 0, 0, 8}

Output : 4 2 12 8 0 0 0 0 0 0

```
import java.util.Arrays;

public class ArrayTransformation {
    public static void main(String[] args) {
        int[] arr1 = {2, 2, 0, 4, 0, 8};
        int[] arr2 = {0, 2, 2, 2, 0, 6, 6, 0, 0, 8};

        transformArray(arr1);
        transformArray(arr2);

        System.out.println("Output for arr1: " + Arrays.toString(arr1));
        System.out.println("Output for arr2: " + Arrays.toString(arr2));
    }

    public static void transformArray(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            if (arr[i] != 0 && arr[i] == arr[i + 1]) {
                arr[i] *= 2;
                arr[i + 1] = 0;
                i++; // Skip the next element as it's already processed
            }
        }

        // Shift all zeros to the end of the array
        int insertIndex = 0;
        for (int num : arr) {
            if (num != 0) {
                arr[insertIndex++] = num;
            }
        }
        while (insertIndex < n) {
            arr[insertIndex++] = 0;
        }
    }
}
```

43 . TWISTED PRIME NUMBER

A number is said to be twisted prime if it is a prime number and reverse of the number is also a prime number.

Input : 97

Output : Twisted Prime Number

Explanation: 97 is a prime number
and its reverse 79 is also a prime
Number.

```
public class TwistedPrimeChecker {
    public static void main(String[] args) {
        int number = 97;
        if (isTwistedPrime(number)) {
            System.out.println(number + " is a twisted prime number.");
        } else {
            System.out.println(number + " is not a twisted prime number.");
        }
    }

    // Function to check if a number is prime
    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    // Function to reverse a number
    public static int reverseNumber(int n) {
        int reverse = 0;
        while (n != 0) {
            int digit = n % 10;
            reverse = reverse * 10 + digit;
            n /= 10;
        }
        return reverse;
    }

    // Function to check if a number is twisted prime
    public static boolean isTwistedPrime(int n) {
        if (!isPrime(n)) {
            return false;
        }
        int reversed = reverseNumber(n);
        return isPrime(reversed);
    }
}
```

44. Given an array A[] and a number x, check for pair in A[] with sum as x.

Eg : Input {1, 2, 4, 3, 5, 6}

SUM : 5

Output : 2 (1, 4) & (2, 3)

```
import java.util.HashSet;

public class PairSumFinder {
    public static void main(String[] args) {
        int[] arr = {1, 2, 4, 3, 5, 6};
        int targetSum = 5;
        findPairsWithSum(arr, targetSum);
    }

    public static void findPairsWithSum(int[] arr, int targetSum) {
        HashSet<Integer> set = new HashSet<>();
        for (int num : arr) {
            int complement = targetSum - num;
            if (set.contains(complement)) {
                System.out.println("Pair found: (" + complement + ", " + num +
                ")");
            }
            set.add(num);
        }
    }
}
```

45. Largest Sum Contiguous Subarray

(Kadane' Algorithm)

```
public class KadaneAlgorithm {
    public static void main(String[] args) {
        int[] arr = { -2, -3, 4, -1, -2, 1, 5, -3 };
        int maxSum = maxSubarraySum(arr);
        System.out.println("Maximum contiguous sum is " + maxSum);
    }

    public static int maxSubarraySum(int[] arr) {
        int maxEndingHere = 0;
        int maxSoFar = Integer.MIN_VALUE;

        for (int i = 0; i < arr.length; i++) {
            maxEndingHere = Math.max(arr[i], maxEndingHere + arr[i]);
            maxSoFar = Math.max(maxSoFar, maxEndingHere);
        }

        return maxSoFar;
    }
}
```

46. Diamond pattern : for given input size -> Here 3

```

*
***
*****
***
*

public class DiamondPattern {
    public static void main(String[] args) {
        int size = 3;
        printDiamondPattern(size);
    }

    public static void printDiamondPattern(int size) {
        // Upper half
        for (int i = 1; i <= size; i++) {
            for (int j = size; j > i; j--) {
                System.out.print(" ");
            }
            for (int k = 1; k <= 2 * i - 1; k++) {
                System.out.print("*");
            }
            System.out.println();
        }

        // Lower half
        for (int i = size - 1; i >= 1; i--) {
            for (int j = size; j > i; j--) {
                System.out.print(" ");
            }
            for (int k = 1; k <= 2 * i - 1; k++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}

```

46. Given a text and a wildcard pattern, implement wildcard pattern matching algorithm that finds if wildcard pattern is matched with text. The matching should cover the entire text (not partial text).

The wildcard pattern can include the characters '?' and '*'

'?' – matches any single character

'*' – Matches any sequence of characters (including the empty sequence)

Example:

Text = "baaabab",

Pattern = "*****ba*****ab",

output : true

Pattern = "baaa?ab", output : true

Pattern = "ba*a?", output : true

Pattern = "a*ab", output : false

```
public class WildcardMatching {
    public static void main(String[] args) {
        String text = "baaabab";
        String pattern1 = "*****ba*****ab";
        String pattern2 = "baaa?ab";
        String pattern3 = "ba*a?";
        String pattern4 = "a*ab";

        System.out.println(isMatch(text, pattern1)); // true
        System.out.println(isMatch(text, pattern2)); // true
        System.out.println(isMatch(text, pattern3)); // true
        System.out.println(isMatch(text, pattern4)); // false
    }

    public static boolean isMatch(String text, String pattern) {
        int n = text.length();
        int m = pattern.length();

        boolean[][] dp = new boolean[n + 1][m + 1];
        dp[0][0] = true;

        for (int j = 1; j <= m; j++) {
            if (pattern.charAt(j - 1) == '*') {
                dp[0][j] = dp[0][j - 1];
            }
        }

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) {
                if (pattern.charAt(j - 1) == '*') {
                    dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
                } else if (pattern.charAt(j - 1) == '?' || text.charAt(i - 1) ==
pattern.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }

        return dp[n][m];
    }
}
```

47. Given an input string and a dictionary of words, find out if the input string can be segmented into a space-separated sequence of dictionary words. See following examples for more details.

Consider the following dictionary

{ i, like, sam, sung, samsung, mobile, ice,
cream, icecream, man, go, mango }

Input: ilike

Output: Yes

The string can be segmented as "i like".

Input: ilikesamsung

Output: Yes

The string can be segmented as "i like samsung"

or "i like sam sung".<>

```
import java.util.*;

public class WordBreak {
    public static void main(String[] args) {
        Set<String> dictionary = new HashSet<>(Arrays.asList("i", "like", "sam",
"sung", "samsung", "mobile", "ice", "cream", "icecream", "man", "go",
"mango"));

        String input1 = "ilike";
        System.out.println(canSegment(input1, dictionary)); // Output: true

        String input2 = "ilikesamsung";
        System.out.println(canSegment(input2, dictionary)); // Output: true
    }

    public static boolean canSegment(String input, Set<String> dictionary) {
        int n = input.length();
        boolean[] dp = new boolean[n + 1];
        dp[0] = true;

        for (int i = 1; i <= n; i++) {
            for (int j = 0; j < i; j++) {
                if (dp[j] && dictionary.contains(input.substring(j, i))) {
                    dp[i] = true;
                    break;
                }
            }
        }

        return dp[n];
    }
}
```

48. Print the following pattern

1

3 2

```

6 5 4
10 9 8 7
10 9 8 7
6 5 4
3 2
1

```

```

public class NumberPattern {
    public static void main(String[] args) {
        printPattern(4);
    }

    public static void printPattern(int n) {
        // Upper half
        for (int i = 1; i <= n; i++) {
            int start = i * (i - 1) / 2 + 1;
            int end = i * (i + 1) / 2;
            for (int j = n; j > i; j--) {
                System.out.print(" ");
            }
            for (int k = start; k <= end; k++) {
                System.out.print(k + " ");
            }
            System.out.println();
        }

        // Lower half
        for (int i = n - 1; i >= 1; i--) {
            int start = i * (i - 1) / 2 + 1;
            int end = i * (i + 1) / 2;
            for (int j = n; j > i; j--) {
                System.out.print(" ");
            }
            for (int k = start; k <= end; k++) {
                System.out.print(k + " ");
            }
            System.out.println();
        }
    }
}

```

49. Given an array as input, The condition is if the number is repeated you must add the number and put the next index value to 0. If the number is 0 print it at the last.

Eg: arr[] = { 0, 2, 2, 2, 0, 6, 6, 0, 8}

Output: 4 2 12 8 0 0 0 0 0 .

```

import java.util.*;

public class ModifyArray {
    public static void main(String[] args) {

```

```

        int[] arr = { 0, 2, 2, 2, 0, 6, 6, 0, 8 };
        int[] modifiedArray = modifyArray(arr);
        for (int num : modifiedArray) {
            System.out.print(num + " ");
        }
    }

    public static int[] modifyArray(int[] arr) {
        int n = arr.length;
        int[] modifiedArray = new int[n];
        Map<Integer, Integer> countMap = new HashMap<>();

        for (int i = 0; i < n; i++) {
            int num = arr[i];
            if (num != 0) {
                if (countMap.containsKey(num)) {
                    modifiedArray[countMap.get(num)] += num;
                    arr[i] = 0;
                } else {
                    countMap.put(num, i);
                }
            }
        }

        int idx = 0;
        for (int num : arr) {
            if (num != 0) {
                modifiedArray[idx++] = num;
            }
        }

        return modifiedArray;
    }
}

```

49. Given two Strings s1 and s2, remove all the characters from s1 which is present in s2.

Input: s1="expErIence", s2="En"

output: s1="exprlece"

```

public class Main {
    public static void main(String[] args) {
        String s1 = "expErIence";
        String s2 = "En";

        String result = removeCharacters(s1, s2);
        System.out.println("Output: " + result);
    }

    public static String removeCharacters(String s1, String s2) {
        StringBuilder sb = new StringBuilder();
    }
}

```

```

        for (int i = 0; i < s1.length(); i++) {
            char c = s1.charAt(i);
            if (s2.indexOf(c) == -1) {
                sb.append(c);
            }
        }

        return sb.toString();
    }
}

```

50. Find the next greater element for each element in given array.

input: array[]={6, 3, 9, 10, 8, 2, 1, 15, 7};

output: {7, 5, 10, 15, 9, 3, 2, _, 8}

If we are solving this question using sorting, we need to use any $O(n \log n)$ sorting algorithm.

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] array = {6, 3, 9, 10, 8, 2, 1, 15, 7};
        int[] result = findNextGreaterElement(array);

        System.out.print("Output: ");
        for (int i = 0; i < result.length; i++) {
            System.out.print(result[i]);
            if (i != result.length - 1) {
                System.out.print(", ");
            }
        }
        System.out.println("");
    }

    public static int[] findNextGreaterElement(int[] array) {
        int n = array.length;
        int[] result = new int[n];
        Stack<Integer> stack = new Stack<>();

        for (int i = 0; i < n; i++) {
            while (!stack.isEmpty() && array[i] > array[stack.peek()]) {
                result[stack.pop()] = i;
            }
            stack.push(i);
        }

        while (!stack.isEmpty()) {
            result[stack.pop()] = -1;
        }
    }
}

```

```

        for (int i = 0; i < n; i++) {
            if (result[i] != -1) {
                result[i] = array[result[i]];
            }
        }

        return result;
    }
}

```

51. Print all distinct permutations of a given string with duplicate characters.

<https://www.geeksforgeeks.org/distinct-permutations-string-set-2>

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        String input = "ABA";
        List<String> permutations = findDistinctPermutations(input);

        System.out.println("Distinct permutations:");
        for (String perm : permutations) {
            System.out.println(perm);
        }
    }

    public static List<String> findDistinctPermutations(String input) {
        List<String> result = new ArrayList<>();
        Map<Character, Integer> frequencyMap = new HashMap<>();

        // Count the frequency of each character
        for (char c : input.toCharArray()) {
            frequencyMap.put(c, frequencyMap.getOrDefault(c, 0) + 1);
        }

        // Convert the frequency map to a character array
        char[] chars = new char[frequencyMap.size()];
        int[] counts = new int[frequencyMap.size()];
        int index = 0;
        for (Map.Entry<Character, Integer> entry : frequencyMap.entrySet()) {
            chars[index] = entry.getKey();
            counts[index] = entry.getValue();
            index++;
        }

        // Create a temporary array to store the current permutation
        char[] temp = new char[input.length()];

        // Generate permutations recursively
        generatePermutations(chars, counts, temp, 0, result);
    }
}

```

```

        return result;
    }

    private static void generatePermutations(char[] chars, int[] counts, char[]
temp, int level, List<String> result) {
        if (level == temp.length) {
            result.add(new String(temp));
            return;
        }

        for (int i = 0; i < chars.length; i++) {
            if (counts[i] == 0) {
                continue;
            }

            temp[level] = chars[i];
            counts[i]--;
            generatePermutations(chars, counts, temp, level + 1, result);
            counts[i]++;
        }
    }
}

```

52. Given a number, find the next smallest palindrome.

```

public class Main {
    public static void main(String[] args) {
        int number = 12321;
        int nextPalindrome = findNextSmallestPalindrome(number);
        System.out.println("Next smallest palindrome: " + nextPalindrome);
    }

    public static int findNextSmallestPalindrome(int number) {
        // Increment the number by 1 to start the search
        number++;

        while (!isPalindrome(number)) {
            number++;
        }

        return number;
    }

    public static boolean isPalindrome(int number) {
        String numStr = String.valueOf(number);
        int i = 0, j = numStr.length() - 1;

        while (i < j) {
            if (numStr.charAt(i) != numStr.charAt(j)) {

```

```

        return false;
    }
    i++;
    j--;
}

return true;
}
}

```

53. Given an array with repeated numbers, Find the top three repeated numbers.

input: array[]={3, 4, 2, 3, 16, 3, 15, 16, 15, 15, 16, 2, 3}

output: 3, 16, 15

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        int[] array = {3, 4, 2, 3, 16, 3, 15, 16, 15, 15, 16, 2, 3};
        List<Integer> topThree = findTopThreeRepeatedNumbers(array);
        System.out.println("Top three repeated numbers: " + topThree);
    }

    public static List<Integer> findTopThreeRepeatedNumbers(int[] array) {
        Map<Integer, Integer> frequencyMap = new HashMap<>();

        for (int num : array) {
            frequencyMap.put(num, frequencyMap.getOrDefault(num, 0) + 1);
        }

        PriorityQueue<Map.Entry<Integer, Integer>> pq = new PriorityQueue<>()
            (a, b) -> b.getValue() - a.getValue();

        pq.addAll(frequencyMap.entrySet());

        List<Integer> topThree = new ArrayList<>();
        for (int i = 0; i < 3 && !pq.isEmpty(); i++) {
            topThree.add(pq.poll().getKey());
        }

        return topThree;
    }
}

```

54. Given two dimensional matrix of integer and print the rectangle can be formed using given indices and also find the sum of the elements in the rectangle

Input: mat[M][N] = {{1, 2, 3, 4, 6}, {5, 3, 8, 1, 2}, {4, 6, 7, 5, 5}, {2, 4, 8, 9, 4}};

index = (2, 0) and (3, 4)

Output:
Rectangle
4 6 7 5 5
2 4 8 9 4
sum 54

```
public class Main {  
    public static void main(String[] args) {  
        int[][] matrix = {  
            {1, 2, 3, 4, 6},  
            {5, 3, 8, 1, 2},  
            {4, 6, 7, 5, 5},  
            {2, 4, 8, 9, 4}  
        };  
  
        int[] topLeft = {2, 0};  
        int[] bottomRight = {3, 4};  
  
        printRectangleAndSum(matrix, topLeft, bottomRight);  
    }  
  
    public static void printRectangleAndSum(int[][] matrix, int[] topLeft, int[]  
bottomRight) {  
        int sum = 0;  
  
        for (int i = topLeft[0]; i <= bottomRight[0]; i++) {  
            for (int j = topLeft[1]; j <= bottomRight[1]; j++) {  
                System.out.print(matrix[i][j] + " ");  
                sum += matrix[i][j];  
            }  
            System.out.println();  
        }  
  
        System.out.println("Sum: " + sum);  
    }  
}
```